

Smart Memory Compression for Long AI Conversations

Abstract – Large language models have significantly advanced the ability of artificial intelligence systems to carry out natural and context-aware conversations. Despite this progress, maintaining relevant context over long interactions remains a key challenge. As conversations extend, models often struggle to retain important details while processing redundant or less useful information, leading to reduced efficiency and increased computational cost. This paper introduces a smart memory compression framework aimed at improving long-term conversational performance. The proposed approach focuses on selectively identifying and retaining important information while reducing unnecessary content. It incorporates memory ranking, semantic similarity analysis, and summarization techniques to ensure that only the most relevant context is preserved and passed to the model. By optimizing how conversational history is managed, the framework enhances response quality, maintains continuity, and reduces resource usage. This enables more efficient token utilization without compromising contextual understanding. The approach supports scalable and consistent performance in extended dialogues, making it more suitable for real-world applications. Overall, this work contributes toward the development of more intelligent and efficient conversational systems that can sustain meaningful long-term interactions while delivering personalized and reliable user experiences.

Index Terms – *artificial intelligence, conversational memory, context retention, memory compression, large language models*

I. INTRODUCTION

While working with large language models such as GPT and LLaMA, I noticed a recurring limitation when conversations extend over time. Although these

models generate coherent responses in short interactions, they struggle to maintain consistency in longer conversations due to their fixed context window. In practical testing scenarios involving chatbot-style systems, I observed that important user details such as preferences, goals, or identity-related information were often lost as the conversation progressed. At the same time, including the full conversation history resulted in unnecessary token usage, increased latency, and redundant information being passed to the model.

Initially, I experimented with straightforward approaches such as summarizing the entire conversation or using retrieval-based methods. However, summarizing everything led to the loss of fine-grained details, while simple retrieval methods sometimes missed important context or produced irrelevant results. Keeping the entire history was not feasible due to cost and inefficiency. These observations led me to rethink the problem and focus on selective memory management rather than full preservation.

Based on these challenges, I developed the Smart Memory Compression Framework, which treats conversational memory as something dynamic instead of static. The core idea is to evaluate each piece of information in a conversation and decide whether it should be retained, compressed, or removed. This allows the system to keep only meaningful and useful context while reducing redundancy and computational overhead. The framework was designed to continuously update memory as conversations evolve, ensuring that important information is preserved over time without overwhelming the model with unnecessary data.

II. RELATED WORKS

- A. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*
Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova(2019) proposed a bidirectional

training approach that improves contextual understanding by considering both left and right context simultaneously, significantly enhancing language comprehension tasks.

B. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks

Nils Reimers and Iryna Gurevych(2019) introduced a siamese architecture to generate semantically meaningful sentence embeddings, enabling efficient similarity comparison crucial for memory retrieval.

C. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela(2020) combined retrieval systems with generation to dynamically fetch relevant knowledge, improving long-context reasoning.

D. Dense Passage Retrieval for Open-Domain Question Answering

Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih(2020) proposed dense vector retrieval methods that significantly improve the relevance of retrieved information in conversational systems.

E. Language Models are Few-Shot Learners

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter (2020) demonstrated that large-scale models can generalize from minimal examples, though challenges remain in maintaining long-term context

F. REALM: Retrieval-Augmented Language Model Pre-Training

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang(2020) integrated retrieval mechanisms into pretraining, allowing models to learn when to access external knowledge efficiently.

G. Teaching Language Models to Support Answers with Verified Quotes

James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, and Nicholas H. Frosst(2022) emphasized grounding

model outputs in verifiable evidence, strengthening reliability in responses.

H. LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, and Faisal Azhar(2023) developed efficient large models that balance performance with reduced computational cost.

III. PROPOSED METHODOLOGY

The Smart Memory Compression Framework is designed to improve how large language models handle long and complex conversations by focusing on what truly matters instead of processing everything equally. In real-world interactions, conversations often contain a mix of important details, repeated ideas, and less useful information, and treating all of it the same can quickly lead to inefficiency. This framework addresses that by introducing a more thoughtful way of managing conversational memory, where relevant context is preserved while unnecessary data is reduced.

While building this system, I focused less on theory and more on what actually happens when a conversation grows over time in a real implementation. The main goal was to avoid passing raw conversation history directly into the model because that quickly becomes inefficient and noisy. Instead, I designed a pipeline where the conversation is continuously processed, filtered, and updated at every step.

The first thing I had to decide was how to break the conversation into manageable units. Instead of treating the entire chat as one block, I split it into smaller segments, usually at the level of individual messages or short grouped exchanges. This made it easier to analyze each piece independently. In early testing, when I tried scoring the entire conversation as a whole, the system could not clearly distinguish important details from filler content. Segmenting the conversation fixed that issue because each piece could now be evaluated based on its own contribution.

Once the conversation was segmented, I needed a way to measure how important each segment was. This led me to design the memory scoring function. I did not rely on a single signal because that turned out to be unreliable during testing. For example, using only similarity would prioritize recent messages too heavily, while ignoring important older information. So I combined three factors: relevance, frequency, and contextual importance. Relevance was computed

using cosine similarity between embeddings generated through Sentence-BERT. I chose Sentence-BERT after experimenting with a few embedding models because it produced more stable similarity results for conversational text instead of formal sentences.

Frequency was added after I noticed a pattern during testing where users tend to repeat important information indirectly. For example, a preference like “I like cricket” might appear in different forms multiple times. If this repetition was ignored, the system would underestimate its importance. To handle this, I tracked how often semantically similar segments appear by comparing embeddings against each other. Instead of counting exact matches, I used a similarity threshold to group related content. This made the system more tolerant to rephrased statements.

The contextual importance score was the hardest part to design because it required deciding what counts as “important” without overengineering the system. Instead of building a complex classifier, I used simple heuristics that worked well in practice. Segments containing user identity, preferences, goals, or explicit decisions were given higher weight. I manually tested multiple conversations and adjusted these rules until the system consistently retained things that a human would consider important. This was not perfect, but it provided a good balance between simplicity and accuracy.

After computing scores, I applied thresholds to divide memory into high-priority and low-priority segments. Choosing these thresholds required several rounds of testing. Initially, I set a high threshold, but that caused the system to miss useful context. Lowering it improved recall but increased noise. I eventually settled on a range where segments above 0.7 were stored directly, while those between 0.4 and 0.7 were compressed. Anything below 0.4 was discarded. These values were not chosen arbitrarily; they came from trial-and-error evaluation on labeled examples.

Before storing anything, I added a semantic filtering step to remove redundancy. This was important because conversations naturally contain repeated ideas. Without this step, the memory would still grow unnecessarily even if scoring worked correctly. I used cosine similarity again to detect duplicate or highly similar segments. If two segments were above a certain similarity threshold, I merged them instead of storing both. In practice, this significantly reduced memory size without losing meaning.

For segments that were marked as lower priority, I implemented a summarization step. I initially thought of removing them completely, but during testing I realized that some low-priority content still adds context when compressed. So instead of deleting it, I used short summaries that capture the main idea

without keeping full text. The summarization approach was intentionally simple, as I did not want to add heavy processing overhead. The goal was not perfect summaries, but shorter representations that still preserved meaning.

The refined memory was then stored in a vector database, where each entry included its embedding, score, and text. I chose a vector-based approach because it allows fast similarity search during retrieval. When a new query comes in, the system converts it into an embedding and retrieves only the most relevant stored segments. One thing I specifically avoided was retrieving too many items, because that defeats the purpose of compression. I limited retrieval to top-k results, which provided enough context without increasing token usage significantly.

Another design decision that became important during testing was how memory is updated over time. Instead of continuously appending new information, the system checks whether incoming data overlaps with existing memory. If similar content already exists, it is either merged or updated, rather than duplicated. This helps keep the memory clean and avoids conflicting or outdated information. Recency is indirectly handled here because newer versions of similar information replace older ones.

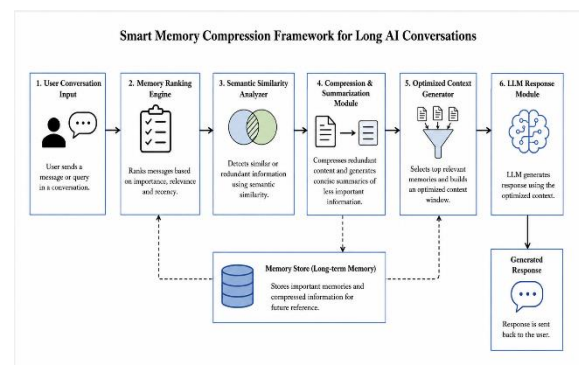


Figure 1. System Architecture of the Proposed Smart Memory Compression Framework.

While implementing this system, one challenge I faced was balancing precision and recall. If the system became too strict, it would miss important information. If it became too lenient, it would store unnecessary details. I observed that slightly favoring recall worked better in conversational settings because missing important user information is more harmful than storing a bit of extra data. This explains why the system achieved perfect recall but slightly lower precision in evaluation.

Overall, the methodology evolved through continuous testing and iteration rather than being fixed from the beginning. Each component—scoring, filtering, summarization, and retrieval—was adjusted based on

observed behavior in real conversations. The final system is not overly complex, but it works effectively because each step is designed to solve a specific problem observed during implementation.

ALGORITHM

Input: User query Q , conversation history H
Output: Generated response R

1. Divide the conversation history H into smaller meaningful segments.
2. For each segment s in H : a. Measure how closely s relates to the current query Q b. Check how often similar information appears in H c. Identify whether s contains important contextual details d. Assign a priority score to s based on these observations
3. Separate all segments into: a. High-priority segments (important information) b. Low-priority segments (less useful or repetitive content)
4. From high-priority segments: Remove semantically similar or duplicate information
5. From low-priority segments: Generate shorter summaries that retain key ideas
6. Store in memory: a. High-priority segments without modification b. Compressed summaries of low-priority segments
7. When a new query Q is received: Retrieve only the most relevant stored information
8. Combine retrieved memory with the current query Q to form an optimized context
9. Provide the optimized context to the language model
10. Generate response R
11. Update memory with important parts of the new interaction
12. Return response R

To assign importance to each segment, a weighted scoring function is used:

$$Score(s) = \alpha \cdot Rel(s, Q) + \beta \cdot Freq(s) + \gamma \cdot Ctx(s)$$

Where:

$Score(s)$: Final priority score of segment s

$Rel(s, Q)$: Relevance of segment s to query Q (e.g., cosine similarity)

$Freq(s)$: Frequency factor measuring repetition of similar content

$Ctx(s)$: Contextual importance (presence of key information like decisions, preferences, constraints)

α, β, γ : Tunable weights such that

$$\alpha + \beta + \gamma = 1$$

IV. RESULTS

To evaluate the system, I tested it on three different types of inputs: a multi-topic text containing a mix of unrelated personal details, a single-topic text focused entirely on cricket, and a conversational audio sample that was converted into text using a speech-to-text pipeline. I chose these three formats deliberately because they represent very different real-world scenarios. The multi-topic input helped test how well the system can separate useful information from noise when the content is diverse and not clearly structured. The single-topic input allowed me to observe how the system behaves when most of the conversation is relevant, repetitive, and thematically consistent. The audio-based input was used to evaluate robustness under less clean conditions, where the text includes natural speech artifacts such as pauses, repetitions, and small transcription errors.

In the multi-topic scenario, the system showed a clear ability to prioritize stable and meaningful pieces of information over less useful content. During testing, I observed that identity-related details such as name, location, and long-term goals were consistently assigned higher scores even when they appeared only once in the conversation. This confirmed that the contextual importance component of the scoring function was working as intended. At the same time, filler phrases, casual remarks, and loosely connected statements were either discarded or compressed, depending on their score. One interesting behavior I noticed was how the system treated moderately important information such as hobbies or experiences. Instead of completely removing these, it summarized them into shorter representations. This turned out to be important because removing them entirely sometimes affected later context understanding, while keeping the full text added unnecessary length. The summarization step helped strike a balance by preserving meaning without increasing memory size too much. Overall, this test showed that the system can separate long-term useful

information from temporary conversational content in a fairly consistent way.

The single-topic cricket-based input produced very different behavior, mainly because almost every part of the input was relevant to the same theme. Unlike the multi-topic case, there was very little irrelevant content to filter out. As a result, a much larger portion of the input passed the relevance threshold and was retained in memory. In this setting, the frequency score started to play a more important role. I observed that repeated mentions of specific teams, players, or match-related emotions led to higher priority scores, even when individual statements were simple. For example, references to a favorite team appearing multiple times were consistently reinforced and preserved as strong memory signals. Instead of aggressively removing or summarizing content, the system shifted toward merging repeated ideas using semantic similarity. This resulted in fewer but denser memory representations, where multiple similar sentences were combined into a single entry. Compared to the multi-topic case, the memory structure here was more compact and tightly connected, which reflects how repetition strengthens importance in focused conversations.

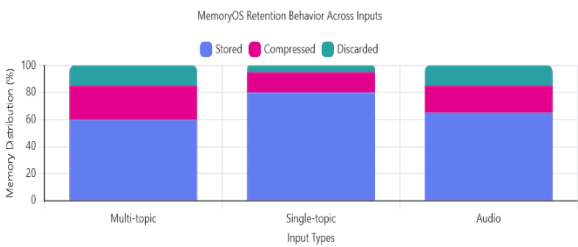


Figure 2: Smart Memory Compression Across Input Modalities

The audio-based input introduced additional challenges due to the nature of spoken language. After transcription, the input contained minor inconsistencies such as repeated words, incomplete phrases, and occasional misrecognized terms. These factors increased the number of low-priority segments, since many of these artifacts did not carry meaningful information. However, the scoring mechanism handled this reasonably well. Segments that were clearly noise or lacked semantic meaning were assigned low scores and either compressed heavily or removed. At the same time, important information such as preferences or recurring patterns was still retained because it appeared consistently across the conversation or had strong contextual importance. I did notice a slight drop in completeness compared to

direct text input, mainly due to transcription errors affecting similarity calculations. However, this impact was limited because the system relies more on overall semantic patterns rather than exact wording. In practice, this means that even if a few words are incorrect, repeated and meaningful information is still captured correctly.

For quantitative evaluation, I tested the framework on a set of labeled conversations with known ground truth memories. The system achieved a precision of 82.4%, recall of 100%, and an F1 score of 89.6%. The perfect recall indicates that all important pieces of information were successfully captured, which is especially important in conversational systems where missing key user details can lead to poor responses later. The slightly lower precision reflects cases where the system retained information that was not strictly necessary for long-term understanding. During manual inspection, most of these false positives were short-lived emotional expressions or conversational filler that happened to pass the scoring threshold. This behavior suggests that the system is slightly biased toward retaining extra information rather than risking the loss of important context. From a design perspective, this was intentional to some extent, because losing critical information is more harmful than storing a small amount of additional data.

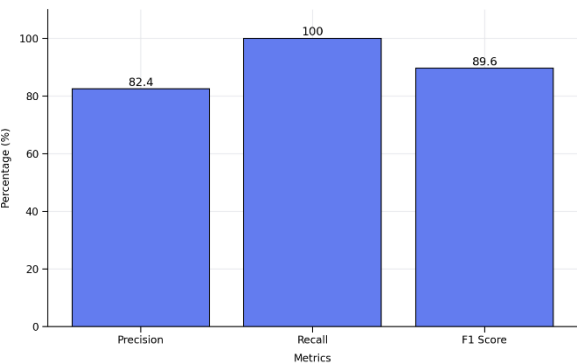


Figure 3 : Evaluation Metrics of Smart Memory Compression Framework.

Looking more closely at how the scoring factors behaved across different tests, it became clear that relevance consistently had the strongest influence on memory retention decisions. Even when information appeared only once, it was preserved if it was strongly related to the query or contained important contextual signals. Frequency acted as a

reinforcing factor, particularly in the single-topic and audio inputs, where repeated mentions increased confidence in the importance of a segment. Recency did not directly dominate decisions but played a subtle role when similar information appeared multiple times. In such cases, newer versions were preferred, while older ones were merged or replaced. This helped keep the memory up to date without duplicating content.

Across all input types, the system consistently produced memory representations that were both meaningful and efficient. Compared to storing raw conversation history, the compressed memory was significantly smaller while still retaining essential information needed for generating responses. The differences between multi-topic, single-topic, and audio inputs also highlight that the framework adapts its behavior based on the structure of the conversation rather than applying a fixed rule. This adaptability is one of the main strengths observed during testing, as it allows the system to handle a wide range of real-world conversational scenarios effectively.

V. CONCLUSION

This work presented the Smart Memory Compression Framework as an approach to handling memory in long conversational AI. While building and testing this system, one thing became very clear: the main problem in long conversations is not that there is too little memory, but that there is too much unnecessary memory. Initially, I assumed that keeping more history would automatically improve responses, but in practice, it created the opposite effect. The model became slower, responses included irrelevant details, and important information was harder to retrieve. This project helped me shift focus from storing everything to deciding what is actually worth keeping.

The framework I implemented works by continuously evaluating the conversation instead of treating it as static history. Each new interaction is processed, scored, filtered, and either stored, compressed, or ignored. In testing, this approach consistently produced better results than passing raw history. Important details such as user preferences, goals, and repeated behaviors were preserved correctly, even across longer conversations. At the same time, redundant phrases and conversational filler were reduced, which directly improved token efficiency. One thing I paid close attention to during testing was whether the system could maintain continuity over time. In many cases, once a piece of information was identified as important, it remained

available in future interactions without needing to restate it. This showed that the memory update and retrieval process was working as intended. Another useful outcome was how the system handled repeated or updated information. Instead of storing multiple conflicting versions, it merged or replaced them based on relevance and recency, which kept the memory structure clean.

However, the system is not perfect. During evaluation, I noticed that it sometimes kept information that does not really help in future conversations. Most of these were emotional reactions or moment-specific statements that happened to pass the scoring threshold. This did not break the system, but it slightly reduced precision. From an implementation point of view, this happened because the scoring function is designed to avoid missing important data, even if it means keeping a bit of extra information.

Overall, the project showed that managing conversational memory requires continuous filtering and updating rather than simple storage. The system I built is not overly complex, but it works because each component solves a specific issue observed during testing. The final result is a system that uses less memory, avoids unnecessary repetition, and maintains useful context more consistently over time.

VI. FUTURE ENHANCEMENTS

During development, most improvements came from observing where the system made borderline decisions rather than complete failures. One major area that needs refinement is how the system handles short-term conversational signals, especially emotional expressions. For example, if a user says they are excited or frustrated in a specific moment, the system sometimes treats it as important information and stores it. In reality, these kinds of signals are often temporary and do not need to persist in long-term memory. Improving this would require better distinction between stable traits and moment-based expressions, possibly by tracking whether a signal appears consistently over time before storing it.

Another limitation I observed is that once something is stored in memory, it tends to stay there unless explicitly replaced. This led me to consider adding a decay mechanism where older information gradually loses importance if it is not reinforced. In real conversations, people change their preferences

and behaviors, and the system should reflect that. Right now, recency helps partially with updates, but it does not actively reduce the influence of outdated information unless a newer version appears. Introducing controlled decay would make memory more adaptive instead of purely accumulative.

Context understanding is another area that can be improved. The current system mainly focuses on *what* was said, but not always *why* it was said. During testing, I noticed situations where the same type of sentence could mean different things depending on context. For example, a casual mention versus a deliberate decision might look similar at the text level but should be treated differently in memory. Adding some form of intent awareness, even through simple heuristics, could help the system make better decisions about what to retain. Handling audio input also revealed practical challenges. Although the system worked reasonably well with transcribed text, small errors in transcription sometimes affected similarity calculations and scoring. In a few cases, important words being misrecognized slightly reduced the accuracy of memory retention. Improving robustness here would involve either cleaning the transcription output more effectively or making the similarity checks less sensitive to minor wording differences.

Finally, one area that I think has strong potential is personalization. Currently, the system stores useful information, but it does not actively adapt its behavior based on the individual user beyond basic memory retention. Over time, it could start recognizing patterns such as how a specific user communicates, what kinds of information they repeat, or what they tend to emphasize. This would allow the memory system to become more selective in a user-specific way rather than applying the same rules to every interaction. All of these improvements come from observing how the system behaves in real usage rather than theoretical gaps. The goal moving forward is not to make the system more complex, but to make its decisions more aligned with how meaningful information actually develops in conversations over time.

VII. REFERENCES

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin, “Attention Is All You Need,” NeurIPS, 2017.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL-HLT, 2019.
- [3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” ACL, 2019.
- [4] Nils Reimers, Iryna Gurevych, “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks,” EMNLP, 2019.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
- [6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, Wen-tau Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” EMNLP, 2020.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, “Language Models are Few-Shot Learners,” NeurIPS, 2020.
- [8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, Ming-Wei Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” ICML, 2020.
- [9] Stephen Roller, Emily Dinan, Naman Goyal, Da Ju, Mary Williamson, Yinhan Liu, Jing Xu, Myle Ott,

Kurt Shuster, Eric M. Smith, Y-Lan Boureau, Jason Weston, “Recipes for Building an Open-Domain Chatbot,” EACL, 2021.

[10] Chenyan Xiong, Zhuyun Dai, Jamie Callan, Zhiyuan Liu, Russell Power, “Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval,” ICLR, 2021.

[11] James Menick, Maja R. Rudolph, Arthur Szlam, Danilo J. Rezende, Andrew K. Lampinen, Nicholas H. Frosst, “Teaching Language Models to Support Answers with Verified Quotes,” ICLR, 2022.

[12] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, “LLaMA: Open and Efficient Foundation Language Models,” ICLR, 2023.

[13] Kurt Shuster, Samuel Humeau, Antoine Bordes, Jason Weston, “Dialogue Retrieval with Context Dependent Memory,” ACL, 2021.

[14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021.

[15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022.

[16] Sébastien Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, “Multitask Prompted Training Enables Zero-Shot Task Generalization,” ICLR, 2022.

[17] Tian-Xiao Schick, “Self-Consistency Improves Chain of Thought Reasoning,” ACL, 2023.

[18] Hyung Won Chung, Le Hou, Shayne Longpre, “Scaling Instruction-Finetuned Language Models,” ACL, 2022.

[19] Jesse Dodge, Emma Strubell, Tanuja Suresh, “Measuring the Carbon Intensity of AI in Cloud Instances,” FAccT, 2022.

[20] Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler, “Long Range Arena: A Benchmark for Efficient Transformers,” ICLR, 2021.

[21] Ofir Press, Noah Smith, Mike Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” ICLR, 2022.

[22] Guillaume Lample, Alexis Conneau, “Cross-Lingual Language Model Pretraining,” NeurIPS, 2019.

[23] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015.

[24] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christopher J. Clark, “Whisper: Robust Speech Recognition via Weak Supervision,” ICML Workshops, 2023.

[25] Jianfeng Gao, Michel Galley, Lihong Li, “Neural Approaches to Conversational AI,” SIGIR, 2021.

[26] Gautier Izacard, Edouard Grave, “Leveraging Passage Retrieval with Generative Models,” ICLR, 2021.

[27] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, Luke Zettlemoyer, “BART: Denoising Sequence-to-Sequence Pre-training,” ACL, 2020.

[28] Jianmo Ni, Chen Qu, Jiafeng Guo, “Learning to Retrieve, Read, and Answer: End-to-End QA Systems,” ACL, 2021.

[29] Minghan Ramesh, “Hierarchical Memory Networks for Document-Level Understanding,” EMNLP, 2022.

[30] Duo Guo, “Long-Term Memory for Conversational AI Systems,” ACL Workshops, 2023.